PROJECT REPORT

RideBoard

A Ride-Sharing Web Application

Instructor: Sir Talha Shahid

Course: Database Systems

Team: Huzaifa Ahmed Bari (24k-0847)
Shoaib Hayat (24k-1028)

Date: 10th May, 2026

1. Introduction

RideBoard is a scheduled carpooling web application that connects drivers who have vacant vehicle seats with riders travelling along the same route. Unlike on-demand commercial ride-hailing services, RideBoard is built around the concept of pre-planned, community-oriented shared rides—where a driver posts a ride for a specific departure time and route, and interested riders can discover and book a seat in advance.

The platform is designed with three primary actors in mind: the **Driver**, who creates and manages rides; the **Rider**, who searches for and books available rides; and the **System**, which automates lifecycle events such as ride expiry and real-time updates.

2. Problem Statement

Urban commuting in Pakistan—particularly in high-density cities such as Karachi—presents significant challenges for students and working professionals:

- **High transport costs:** Commercial ride-hailing platforms (Careem, Uber) charge per-kilometre fares, making daily commuting expensive for regular users.
- **Underutilised vehicle capacity:** A large proportion of privately owned vehicles carry only the driver, with multiple empty seats—a significant resource waste.
- **Lack of trusted peer-to-peer carpooling:** No mainstream platform exists that enables scheduled, community-based seat sharing among commuters who routinely travel the same route.
- **Traffic congestion:** Low vehicle occupancy rates directly contribute to peak-hour congestion on city roads.
- **No communication infrastructure:** Informal carpooling arrangements lack integrated tools for driver–rider coordination and accountability.

3. Objectives

The key objectives of the RideBoard project are:

- Build a reliable, browser-accessible web application for scheduling and booking carpooled rides.
- Provide an intuitive, map-based interface for ride creation and discovery.
- Enable real-time seat booking with instant availability feedback.
- Facilitate direct communication between ride participants through a per-ride group chat.
- Establish community trust through a post-ride user rating and review system.
- Automate the full ride lifecycle—from active through full to expired—using database-level scheduling.
- Deliver a responsive, cross-browser-compatible UI accessible on both desktop and mobile devices.

4. System Architecture

RideBoard follows a three-tier client-server architecture. The table below describes each tier, its components, and its responsibilities:

Tier	Component	Responsibilities
Client	React.js · Leaflet.js	UI, interactive map, real-time subscriptions
Application	Node.js / Express.js	REST API routing, request validation, Supabase proxy
Data	Supabase (PostgreSQL)	Relational storage, Auth (JWT), Row Level Security, real-time WebSocket feeds, pg_cron scheduler
Geospatial	OSRM · Nominatim	Route polyline generation (GeoJSON) and address geocoding fallback via OpenStreetMap

Table 1: Three-Tier Architecture Overview

4.1 Frontend (Client Tier)

The frontend is built with React.js. It communicates with the backend via REST API calls over HTTPS. Key frontend responsibilities include:

- Rendering the interactive Leaflet.js map for ride origin/destination selection.
- Displaying route polylines generated by OSRM.
- Subscribing to Supabase real-time channels for live chat messages and seat count updates.

4.2 Backend (Application Tier)

The Node.js/Express.js server acts as a thin API layer. It handles:

- Request validation and routing.
- Proxying authenticated requests to Supabase.
- Encapsulating business logic (e.g., seat count management, booking state transitions).
- Serving the API to the React frontend.

4.3 Data Tier (Supabase)

Supabase provides the full data persistence layer with the following components:

- **PostgreSQL database** — relational storage for Users, Rides, Bookings, and Messages.
- **Supabase Auth** — email/password authentication; issues JWTs consumed by the frontend and backend.

- **Row Level Security (RLS)** — database-level access control ensuring users only read/write their own data.
- **Real-time subscriptions** — WebSocket-based change feeds used for live chat delivery and seat availability updates.
- **pg_cron** — a PostgreSQL extension running a scheduled job every minute to expire past-due rides.

4.4 External Services

- **OSRM (Open Source Routing Machine)** — generates route polylines (GeoJSON) between two coordinate pairs. Queried by the backend during ride creation.
- **Nominatim** — OpenStreetMap-based geocoding used as a fallback to convert text addresses into latitude/longitude coordinates.

5. Technology Stack

Layer	Technology
Frontend Framework	React.js
Frontend Styling	Vanilla CSS
Map Rendering	Leaflet.js
Backend Runtime	Node.js
Backend Framework	Express.js
Database & Auth	Supabase (PostgreSQL)
Route Calculation	OSRM
Geocoding	Nominatim
Scheduling	pg_cron
Version Control	Git / GitHub

Table 2: Technology Stack

6. Database Design

6.1 Entity Relationship Overview

The database contains four core tables with the following relationships:

- A User can post many Rides (as a driver).
- A Ride can have many Bookings.
- A User can have many Bookings (as a rider).
- A Ride has many Messages (its group chat).
- A User sends many Messages.

6.2 Table Definitions

6.2.1 Users

Field	Type	Constraints	Description
id	UUID	PRIMARY KEY	Linked to Supabase Auth UID
name	TEXT	NOT NULL	Full name
email	TEXT	UNIQUE, NOT NULL	Email address
phone	TEXT	—	Contact number
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time

Table 3: Users Table

6.2.2 Rides

Field	Type	Constraints	Description
id	UUID	PRIMARY KEY	Auto-generated
poster_id	UUID	FK → Users	The driver
origin_address	TEXT	NOT NULL	Human-readable origin
destination_address	TEXT	NOT NULL	Human-readable destination
origin_lat	FLOAT	NOT NULL	Origin latitude
origin_lng	FLOAT	NOT NULL	Origin longitude
destination_lat	FLOAT	NOT NULL	Destination latitude
destination_lng	FLOAT	NOT NULL	Destination longitude
start_time	TIMESTAMP	NOT NULL	Scheduled departure
fare	NUMERIC	NOT NULL	Fare per seat (PKR)
total_seats	INTEGER	CHECK (1–8)	Max capacity
seats_remaining	INTEGER	NOT NULL	Current availability
status	TEXT	DEFAULT ‘active’	active / full / expired
created_at	TIMESTAMP	DEFAULT NOW()	Record creation time

Table 4: Rides Table

6.2.3 Bookings

Field	Type	Constraints	Description
id	UUID	PRIMARY KEY	Auto-generated
ride_id	UUID	FK → Rides	The booked ride
rider_id	UUID	FK → Users	The booking rider
status	TEXT	DEFAULT 'confirmed'	confirmed / cancelled
created_at	TIMESTAMP	DEFAULT NOW()	Booking timestamp

Table 5: Bookings Table

6.2.4 Messages

Field	Type	Constraints	Description
id	UUID	PRIMARY KEY	Auto-generated
ride_id	UUID	FK → Rides	The ride's group chat
sender_id	UUID	FK → Users	Message author
content	TEXT	NOT NULL	Message body
created_at	TIMESTAMP	DEFAULT NOW()	Send timestamp

Table 6: Messages Table

6.2.5 Reviews

Field	Type	Constraints	Description
id	UUID	PRIMARY KEY	Auto-generated
ride_id	UUID	FK → Rides	The ride context
reviewer_id	UUID	FK → Users	User submitting the review
reviewee_id	UUID	FK → Users	User being reviewed
rating	INTEGER	CHECK (1–5)	Star rating from 1 to 5
comment	TEXT	—	Optional written feedback
created_at	TIMESTAMP	DEFAULT NOW()	Review submission timestamp

Table 7: Reviews Table

6.3 Row Level Security Policies

RLS is enabled on all tables. Key policy rules:

- **Users:** A user can only `SELECT` and `UPDATE` their own row (`auth.uid() = id`).
- **Rides:** Any authenticated user can `SELECT` active rides. Only the poster can `UPDATE` or `DELETE` their own ride.
- **Bookings:** A rider can `INSERT` a booking for themselves. Only the rider can `UPDATE` their own booking status.
- **Messages:** Any confirmed participant of a ride (driver or confirmed rider) can `INSERT` and `SELECT` messages for that ride.

7. Implementation Details

7.1 Ride Posting Flow

When a driver creates a ride:

1. The frontend renders a Leaflet.js map with clickable marker placement for origin and destination.
2. Once both markers are placed, the frontend requests the route from OSRM.
3. The returned GeoJSON polyline is drawn on the map using Leaflet's `L.geoJSON()`.
4. The driver completes the form (time, fare, seats) and submits.
5. The Express backend receives the ride data, validates it, and inserts a new row into the `rides` table via the Supabase client.
6. Supabase real-time immediately propagates the new ride to any active search subscriptions.

7.2 Real-Time Seat Management

When a rider books a seat, the backend uses a PostgreSQL function to atomically update the seat count. This prevents race conditions where two riders simultaneously attempt to book the last seat:

```
UPDATE rides
SET seats_remaining = seats_remaining - 1,
    status = CASE WHEN seats_remaining - 1 = 0
                  THEN 'full'
                  ELSE status
             END
WHERE id = $ride_id
  AND seats_remaining > 0
RETURNING seats_remaining, status;
```

7.3 Automated Ride Expiry

The `pg_cron` job is registered in Supabase to run every minute. It automatically transitions any ride whose departure time has passed to an `expired` status:

```
SELECT cron.schedule(
  'expire-past-rides',
  '* * * * *',
  $$
    UPDATE rides
    SET status = 'expired'
    WHERE start_time < NOW()
      AND status IN ('active', 'full');
  $$
);
```

This job runs server-side every minute.

8. Conclusion

RideBoard successfully delivers a full-stack, scheduled carpooling web application that addresses a real and prevalent commuting problem. The system demonstrates a cohesive use of modern web technologies—React.js, Node.js/Express.js, and Supabase—to deliver key features including map-based ride creation, real-time seat management, group chat, and automated ride lifecycle management.

9. References

- Supabase Documentation — https://supabase.com/docs
- OSRM Project — http://project-osrm.org
- Nominatim / OpenStreetMap — https://nominatim.openstreetmap.org/
- Leaflet.js Documentation — https://leafletjs.com/
- React.js Documentation — https://react.dev
- Express.js Documentation — https://expressjs.com/
- PostgreSQL pg_cron Extension — https://github.com/citusdata/pg_cron
- RideBoard GitHub Repository — https://github.com/HuzaifaAhmedBari/Ride-Board